1. Constructor in C#

A constructor is a special method that runs automatically when an object is created.

Example

```
class Student
{
   public string name;

   // Constructor
   public Student()
   {
      name = "Rahul";
      Console.WriteLine("Constructor called");
   }
}

class Program
{
   static void Main()
   {
      Student s = new Student();
      Console.WriteLine(s.name);
   }
}
```

Output:

```
Constructor called
Rahul
```

Important points

Constructor has the same name as the class.

It has no return type, not even void.

It is automatically called when an object is created.

Constructors are commonly used to initialize fields.

Parameterized constructor

```
class Student
{
   public string name;
   public int age;

   public Student(string name, int age)
   {
      this.name = name;
      this.age = age;
   }
}
```

Usage:

```
Student s = new Student("Rahul", 20);
```

2. this Keyword in C#

this refers to the current object.

It is commonly used when a constructor parameter has the same name as a class field.

```
class Student
```

```
{
    string name;
    int age;

    public Student(string name, int age)
    {
        this.name = name;
        this.age = age;
    }
}
```

Here:

this.name = name;

means:

this.name → class field

name → constructor parameter

Without this, C# would understand both name references as the parameter.

Another use of this

You can use this to call another constructor:

```
class Student
{
    string name;
    int age;
```

```
    public Student() : this("Unknown", 0)
    {
    }

    public Student(string name, int age)
    {
        this.name = name;
        this.age = age;
    }
}
```

3. Destructor in C#

A destructor is used when an object is being cleaned up by the Garbage Collector.

Syntax:

```
class Student
{
    ~Student()
    {
        Console.WriteLine("Destructor called");
    }
}
```

Important points:

Destructor has the same name as the class with ~.

It cannot have parameters.

It cannot have an access modifier.

You cannot call a destructor directly.

In C#, destructors are actually implemented as finalizers.

You generally don't use destructors for normal resource cleanup. For resources such as files or database connections, IDisposable and Dispose() are normally preferred.

4. Inheritance in C#

Inheritance allows one class to acquire members of another class.

The existing class is called the base/parent class.

The new class is called the derived/child class.

Example

```
class Animal
{
   public void Eat()
   {
      Console.WriteLine("Animal is eating");
   }
}

class Dog : Animal
{
   public void Bark()
   {
      Console.WriteLine("Dog is barking");
   }
}
```

Usage:

```
Dog d = new Dog();

d.Eat();   // inherited from Animal
d.Bark();  // Dog's own method
```

Output:

```
Animal is eating
Dog is barking
```

The : symbol means inheritance:

```
class Dog : Animal
```

5. Constructor and Inheritance

When you create an object of a derived class, the base-class constructor executes first, followed by the derived-class constructor.

```
class Animal
{
   public Animal()
   {
      Console.WriteLine("Animal constructor");
   }
}

class Dog : Animal
{
```

```
    public Dog()
    {
        Console.WriteLine("Dog constructor");
    }
}

Dog d = new Dog();
```

Output:

```
Animal constructor
Dog constructor
```

Calling a specific base constructor

Use base:

```
class Animal
{
    public Animal(string name)
    {
        Console.WriteLine("Animal: " + name);
    }
}

class Dog : Animal
{
    public Dog(string name) : base(name)
    {
        Console.WriteLine("Dog constructor");
    }
}
```

```
Dog d = new Dog("Tommy");
```

Output:

```
Animal: Tommy
Dog constructor
////////////////////////////////////
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp27
{
  public  class A
   {
    public void d()
      {
         Console.WriteLine("hello i am class A");
      }
   }
   internal class Program : A
   {
      void disp1()
      {
         Console.WriteLine("hello i am class Program");
      }
      /* public  Program()
        {
```

```
        String name = "alvin";
        Console.WriteLine(name);
    }
  int x, y;
public Program(int a , int b)
  {
     x = a; y = b;
  }
  void disp()
  {
     int add = x + y;
     Console.WriteLine(add);
  }

  int a,b ;
  public Program(int a, int b)
  {
     this.a = a; this.b = b;
  }
  void disp()
  {
     int add = this.a + this.b;
     Console.WriteLine(add);
  }*/


  static void Main(string[] args)
  {
     Program p1 = new Program();
     p1.disp1();
     p1.d();
```

```
        /*   p1.disp();
          Console.ReadKey();*/
        Console.ReadKey();
      }
   }
}
```

//////////////////////////////////////

C# Practical Questions – Constructors, Parameterized Constructors & this Keyword

Student Class

Create a C# program to define a Student class with a default constructor that initializes the student name and age. Display the details.

Employee Class

Create a class Employee with a parameterized constructor that accepts employee ID, name, and salary. Display the employee details.

Book Class

Create a Book class with a parameterized constructor to initialize book title, author, and price. Display the book information.

Using this Keyword

Create a Person class with data members name and age. Use the this keyword in the parameterized constructor to initialize these values.

Rectangle Class

Create a Rectangle class with length and breadth. Use a parameterized constructor to initialize them and calculate the area.

Product Class

Create a Product class with product ID, product name, and price. Initialize these values using a parameterized constructor and display them.

Default and Parameterized Constructor

Create a Car class containing model name and price. Implement both a default constructor and a parameterized constructor. Display the values using both constructors.

Bank Account

Create a BankAccount class with account number, account holder name, and balance. Use a parameterized constructor and the this keyword to initialize the data members.

College Student

Create a CollegeStudent class with roll number, name, and course. Use a parameterized constructor with the this keyword to initialize and display the student details.

Calculate Total Marks

Create a Student class with name and marks of three subjects. Use a parameterized constructor and the this keyword to initialize the values. Calculate and display the total marks.